

```

1 # 06 MATRIZ_SAM
2
3 # Este archivo construye el archivo que se integra a sam seleccionado los archivos de entrada
4
5 import sys
6 import csv
7 from pathlib import Path
8 import pandas as pd
9
10 def construir_archivo_salida(matriz_weekday_path, matriz_weekend_path, tod_path, archivo_salida_path, year=2025):
11     # --- Leer matrices (12x24) como enteros ---
12     mw = pd.read_csv(matriz_weekday_path, header=None).astype(int).values.tolist()
13     me = pd.read_csv(matriz_weekend_path, header=None).astype(int).values.tolist()
14
15     # --- Leer columna H del archivo TOD y tomar H2..H10 ---
16     # Intento 1: con encabezado y columna "TOD"
17     df_tod = pd.read_csv(tod_path)
18     tod_factors = None
19
20     if "TOD" in df_tod.columns:
21         # H2..H10 => primeras 9 filas de la columna "TOD"
22         tod_factors = df_tod["TOD"].iloc[0:9].astype(float).tolist()
23     else:
24         # Intento 2: sin encabezado -> columna H es índice 7, filas 1..9 -> iloc[1:10, 7]
25         df_tod = pd.read_csv(tod_path, header=None)
26         if df_tod.shape[1] <= 7 or df_tod.shape[0] < 10:
27             raise ValueError("El archivo TOD no tiene la columna H o no tiene al menos 10 filas.")
28         tod_factors = df_tod.iloc[1:10, 7].astype(float).tolist()
29
30     # Mapeo 1..9 -> valores (H2..H10)
31     tod_map = {i + 1: tod_factors[i] for i in range(9)}
32
33     # --- Helpers ---
34     def fmt_2dec(x):
35         # 2 decimales; elimina ceros/punto sobrantes (0.60 -> 0.6, 1.00 -> 1)
36         return f"{x:.2f}".rstrip("0").rstrip(".")
37
38     def matrix_to_sched_str(matrix_12x24):
39         # "[a;b;...;z]" por cada fila (24) y concatena 12 filas -> ...][...
40         filas = []
41         for fila in matrix_12x24:
42             filas.append("[" + ",".join(str(int(v)) for v in fila) + "]")
43         return "".join(filas)
44
45     weekday_sched = matrix_to_sched_str(mw)
46     weekend_sched = matrix_to_sched_str(me)
47
48     # --- dispatch_factors_ts FIJO (se usa SIEMPRE esta serie) ---
49     DISPATCH_TS_FIXED = ""
50     0.7;0.7;0.7;0.7;0.7;0.8;0.8;1.1;1.1;1.1;1.1;1.1;1.1;1.1;1.1;1.1;1.1;1.1;0.8;0.8;0.8;0.7;0.7;0.7;0.7;0.8;0.8;1.1;1.1;1.1;1.1;
51     """.strip()
52
53     # Normaliza: quita saltos y espacios
54     lines = []
55     for line in DISPATCH_TS_FIXED.splitlines():
56         t = line.strip()
57         if t and not t.startswith("#"):
58             lines.append(t)
59     disp_ts_str = ",".join(x.strip() for x in ",".join(lines).split(",") if x.strip())
60
61     # PPA TOD factors con 2 decimales exactos (orden 1..9)
62     dispatch_tod_factors_str = ",".join(fmt_2dec(tod_map[i]) for i in range(1, 10))
63
64     # --- Escribir salida con la MISMA estructura que Untitled.csv ---
65     out_path = Path(archivo_salida_path)
66     with open(out_path, "w", newline="", encoding="utf-8") as f:
67         w = csv.writer(f)
68
69         # Orden y etiquetas como en tu ejemplo; cada factor toma H2..H10 según corresponda
70         w.writerow(['lib_dispatch_factor7', fmt_2dec(tod_map[7]), 'TOD factor 7 from TOD library'])
71         w.writerow(['ppa_multiplier_model', '0', 'TOD factor mode'])
72         w.writerow(['lib_dispatch_factor2', fmt_2dec(tod_map[2]), 'TOD factor 2 from TOD library'])
73         w.writerow(['lib_dispatch_factor8', fmt_2dec(tod_map[8]), 'TOD factor 8 from TOD library'])
74         w.writerow(['lib_dispatch_factor1', fmt_2dec(tod_map[1]), 'TOD factor 1 from TOD library'])
75         w.writerow(['lib_dispatch_factor6', fmt_2dec(tod_map[6]), 'TOD factor 6 from TOD library'])
76         w.writerow(['tod_library', 'Uniform Dispatch', 'TOD factors and schedule library'])
77         w.writerow(['lib_dispatch_factor3', fmt_2dec(tod_map[3]), 'TOD factor 3 from TOD library'])
78
79         # Schedules
80         w.writerow(['lib_dispatch_sched_weekend', weekend_sched, 'TOD weekend schedule from TOD library'])
81         w.writerow(['dispatch_sched_weekday', weekday_sched, 'TOD schedule weekday'])
82         w.writerow(['lib_dispatch_sched_weekday', weekday_sched, 'TOD weekday schedule from TOD library'])
83
84         # Factores agregados
85         w.writerow(['dispatch_tod_factors', dispatch_tod_factors_str, 'PPA TOD factors'])
86         w.writerow(['dispatch_factors_ts', disp_ts_str, 'TOD factors by time step'])

```

```

87
88     # Resto de factores
89     w.writerow(['lib_dispatch_factor5', fmt_2dec(tod_map[5]), 'TOD factor 5 from TOD library'])
90     w.writerow(['lib_dispatch_factor4', fmt_2dec(tod_map[4]), 'TOD factor 4 from TOD library'])
91     w.writerow(['lib_dispatch_factor9', fmt_2dec(tod_map[9]), 'TOD factor 9 from TOD library'])
92
93     # Repite weekend schedule al final (formato del ejemplo)
94     w.writerow(['dispatch_sched_weekend', weekend_sched, 'TOD schedule weekend'])
95
96     return str(out_path)
97
98
99 if __name__ == "__main__":
100     # Directorio del propio script
101     base = Path(__file__).parent.resolve()
102
103     # Toma la clave del nodo desde argumentos (como la pasa tu runner)
104     clave_nodo = sys.argv[1] if len(sys.argv) >= 2 else "03VDR-230"
105
106     # Rutas de entrada/salida
107     matriz_weekday_path = base / f"Matriz_Weekday_{clave_nodo}.csv"
108     matriz_weekend_path = base / f"Matriz_Weekend_{clave_nodo}.csv"
109     tod_path = base / f"TOD_{clave_nodo}.csv"
110     archivo_salida_path = base / "Untitled.csv" # o base / f"Untitled_{clave_nodo}.csv"
111
112     # Verificación de archivos
113     faltantes = [p for p in [matriz_weekday_path, matriz_weekend_path, tod_path] if not p.exists()]
114     if faltantes:
115         print("No se encontraron los siguientes archivos requeridos:")
116         for p in faltantes:
117             print(" -", p)
118         sys.exit(1)
119
120     print(" Entradas:")
121     print(" -", matriz_weekday_path)
122     print(" -", matriz_weekend_path)
123     print(" -", tod_path)
124     print(" Salida:", archivo_salida_path)
125
126     salida = construir_archivo_salida(
127         str(matriz_weekday_path),
128         str(matriz_weekend_path),
129         str(tod_path),
130         str(archivo_salida_path),
131         year=2025
132     )
133     print("Generado:", salida)

```